

Francisco Acuña - 20220565

Isaac Cyrman - 20220289

Rodrigo Reyes - 20200090

Documentación

Funcionamiento de la interrupción del timer

El sistema configura un temporizador (timer) en modo periódico, de modo que, al cargarse un valor de cuenta (calculado como el número de segundos multiplicado por 1.000.000 para simular un reloj de 1 MHz), éste decrece hasta llegar a cero. Al terminar la cuenta, el timer genera una interrupción (IRQ). La interrupción es detectada mediante el registro T_MIS, el cual indica si existe una interrupción pendiente. Cuando se produce la IRQ, se ejecuta la función `timer_irq_handler`, que limpia la bandera de interrupción escribiendo en T_INTCLR. Además, se notifica al controlador de interrupciones (VIC) que la IRQ ha sido atendida mediante la escritura de 0 en el registro VIC_ADDR. Para evitar saturar la terminal, se ha programado que el mensaje de "Timer IRQ Fired" se imprima únicamente la primera vez.

Descripción de cada función modificada

a) `timer_init(int segs)`

- Esta función se encarga de inicializar el timer con un valor de cuenta basado en el número de segundos indicado ($\text{segs} \times 1.000.000$).
- Configura el timer en modo periódico activándolo y habilitando su generación de interrupciones (definiendo bits en T_CTRL).
- Habilita la línea de interrupción correspondiente en el VIC mediante la escritura en VIC_ENABLE.
- Imprime un mensaje de depuración indicando si la configuración del VIC fue correcta (este mensaje se muestra una única vez durante la inicialización).

b) `timer_irq_handler(void)`

- Esta es la rutina que se invoca cuando se produce una interrupción del timer.
- Verifica mediante el registro T_MIS si la interrupción está pendiente.
- Utiliza una variable estática para asegurarse de que el mensaje "Timer IRQ Fired" se imprima solo la primera vez que se dispara la interrupción.
- Limpia la bandera de interrupción escribiendo en T_INTCLR y notifica al VIC que la IRQ ha sido atendida escribiendo 0 en VIC_ADDR.

c) irq_enable(void)

- Esta función habilita las IRQ a nivel del CPU.
- Se implementa en ensamblador dentro de una función en C, donde se modifica el registro CPSR para limpiar el bit que deshabilita las interrupciones (bit 7).
- Esto permite que el CPU acepte interrupciones.

d) delay(int ciclos)

- Función que implementa un retardo basado en busy-wait (espera activa), consumiendo ciclos de CPU a través de la instrucción "nop".
- Se utiliza en el bucle principal para evitar un uso excesivo de la CPU sin necesidad de imprimir demasiados mensajes.

e) main(void)

- Función principal del sistema, en la que se inicializa la UART y se muestran mensajes que indican el inicio del sistema.
- Se asigna el callback de interrupción (timer_callback) a la función timer_irq_handler.
- Se inicializa el timer para generar una IRQ cada 2 segundos llamando a timer_init(2).
- Se habilitan las interrupciones a nivel de CPU mediante la función irq_enable.
- Finalmente, entra en un bucle infinito en el que se utiliza delay para evitar que el CPU se sature.

f) root.s

- Archivo de ensamblador que define el punto de entrada (_start) del sistema.
- Inicializa el stack y llama a la función main() en C.
- Contiene la rutina irq_handler, la cual se encarga de:
 - Ajustar el registro LR para que apunte a la instrucción correcta tras la interrupción.
 - Guardar el estado de los registros (r0 a r12 y LR) en la pila.
 - Llamar a timer_irq_handler para atender la interrupción en C.
 - Restaurar los registros y retornar de la interrupción (regresando a la ejecución normal).

g) uart.c y uart.h

- Estos archivos proporcionan funciones para inicializar la UART (uart_init) y para enviar caracteres y cadenas (uart_putc y uart_puts).
- Se utilizan para imprimir mensajes de depuración y estado en la consola, facilitando la observación de la ejecución en QEMU o hardware real.

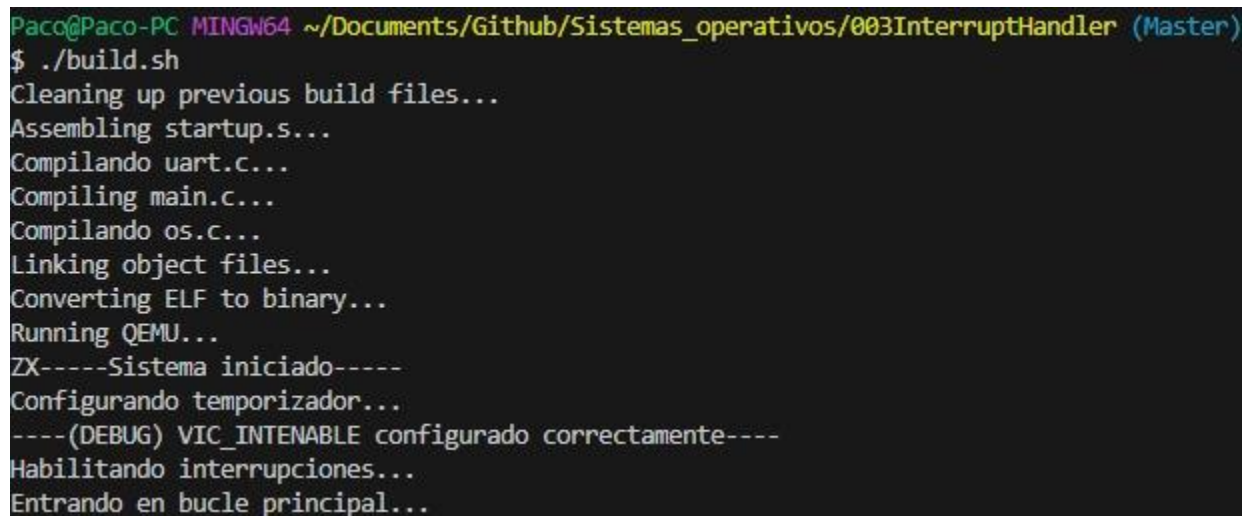
Manejo de las IRQs en el sistema

El sistema gestiona las IRQs a través de la siguiente secuencia:

- Al inicio, el archivo root.s establece la tabla de vectores y define _start, el cual configura el stack e invoca main().
- En main(), se inicializa la UART y se configura el timer para que genere una interrupción cada 2 segundos.
- La función irq_enable() es llamada para habilitar las interrupciones a nivel de CPU, permitiendo que el procesador acepte señales de IRQ.
- Cuando el timer termina su cuenta, se activa una IRQ.
- El VIC (Vector Interrupt Controller) enruta la interrupción al CPU, el cual transfiere el control a la rutina irq_handler definida en root.s.
- En irq_handler, se guarda el estado de los registros y se invoca timer_irq_handler, que se encarga de atender la interrupción: verifica el estado, imprime el mensaje (solo la primera vez), limpia la bandera y notifica al VIC.
- Tras el retorno de timer_irq_handler, irq_handler restaura el contexto y la ejecución del programa continúa normalmente.

Esta cadena de acciones garantiza que la interrupción del timer se maneje de manera eficiente y que el sistema retome su funcionamiento habitual después de cada evento de IRQ.

Screenshots:



```
Paco@Paco-PC MINGW64 ~/Documents/Github/Sistemas_operativos/003InterruptHandler (Master)
$ ./build.sh
Cleaning up previous build files...
Assembling startup.s...
Compilando uart.c...
Compiling main.c...
Compilando os.c...
Linking object files...
Converting ELF to binary...
Running QEMU...
ZX-----Sistema iniciado-----
Configurando temporizador...
----(DEBUG) VIC_INTENABLE configurado correctamente----
Habilitando interrupciones...
Entrando en bucle principal...
```

[illegible]